Procedural generation of forests using L-systems and Poisson-Disk Sampling

DH2323 Computer Graphics and Interaction

Final project

Linus Engvall

KTH Royal Institute of Technology

EECS School of Electrical Engineering and Computer Science

19991003-0056

lengv@kth.se

Matilda Jansson

KTH Royal Institute of Technology

EECS School of Electrical Engineering and Computer Science

20010503-2841

matilja@kth.se

Abstract

This project explores the combination of L-systems and Poisson-Disk Sampling for efficient and visually plausible procedural forest generation. By using Fast Poisson-Disk Sampling to control tree placement and deterministic L-systems to generate tree structures, the system enables the creation of large-scale forests with minimal manual input. The implementation focuses on modularity and interactivity, allowing users to adjust parameters and instantly visualize changes in Unity. The project demonstrates how two well-established techniques can be combined to achieve scalable, flexible, and aesthetically convincing results. The resulting system is lightweight and extendable, offering a practical tool for procedural content creation.

Figure 1: Visualization of trees modeled by L-systems and placed with natural spacing using Poisson-Disk Sampling

1 Introduction

Procedural forest generation is a rapidly growing field not only in game development but also in scientific research and ecosystem modeling. Manual placement of trees in virtual environments is labor-intensive and may lack ecological realism, whereas procedural methods can drastically reduce development time while producing more natural distributions [1].

In this project, we focus on two core techniques: Poisson-disk sampling, for generating natural and evenly spaced tree placements, and L-systems, for modeling the internal structure of trees. Poisson-disk sampling allows for spatial distributions that mimic ecological spacing, avoiding both clustering and overly regular grids. Meanwhile, L-systems offer a compact and flexible way to model and visualize plant growth patterns through rule-based branching structures.

Together, these methods allow for scalable and visually plausible forest generation without relying on hand-crafted assets or extensive manual work. This approach is not only computationally efficient but also aligns with ecological principles, making it useful in both artistic and scientific applications.

2 Background

2.1 L-Systems

L-systems, or Lindenmayer systems, are a mathematical formalism introduced by biologist Aristid Lindenmayer in 1968 to model the growth of multicellular organisms. They are based on rewriting rules, where a simple initial string (called the axiom) is repeatedly transformed using a set of production rules. Unlike traditional grammars, where rules are applied one at a time, L-systems apply all rules in parallel, reflecting biological processes like simultaneous cell division. While originally developed for theoretical biology, L-systems later found wide applications in computer graphics, particularly in the generation of fractals and realistic plant structures.

A common variant is the D0L-system (Deterministic 0-context L-system), where the rewriting is deterministic (no randomness) and context-free (each symbol is replaced independently of its neighbors). This simplicity makes D0L-systems particularly suited for structured, recursive designs like fractals and branching forms. [2]

2.2 Poisson-Disk Sampling

Poisson-Disk Sampling is a method for generating points that are randomly distributed while maintaining a minimum distance between each point. This avoids clustering and gaps, and produces more uniform distributions compared to purely random uniform sampling. The algorithm is the most popular sampling method to distribute with unbiased randomness and even distribution at the same time [3].

The concept of Poisson-disk sampling was initially explored using the dart throwing technique, introduced by Cook [4], which ensures that no two points are closer than a minimum radius. More recent comprehensive surveys provide an overview of blue-noise sampling properties and algorithms, highlighting the balance between randomness and spatial uniformity that Poisson-disk methods offer.

Poisson-disk sampling is widely used in computer graphics, spatial analysis, and procedural generation due to its ability to produce blue-noise characteristics. This makes it ideal for applications such as rendering, stippling, texture synthesis, object placement in virtual environments, and simulation tasks [5].

2.2.1 Fast Poisson-Disk Sampling

Traditional Poisson-disk sampling methods, such as the dart throwing technique, tend to be inefficient, particularly when applied in higher dimensions [6]. Fast Poisson-Disk Sampling, also known as Bridson's algorithm, presents an improved approach that efficiently generates Poisson disk samples across multiple dimensions. By employing a background grid for fast spatial searches and maintaining an active list of sample points, the algorithm achieves a time complexity of O(N). This advancement addresses the limitations of earlier methods and expands the practical use of Poisson-disk sampling in complex graphics applications.

While Bridson's fast Poisson-Disk Sampling algorithm's primary advantage lies in its computational speed and scalability to higher dimensions, the method

is known to produce samples that are not strictly maximal, meaning the points do not cover the domain as densely as possible, and it introduces some bias in the distribution. This means that for cases where statistical guarantees or maximum disk packing is needed, fast Poisson-Disk Sampling may not be the ideal choice. Despite these limitations, Bridson's algorithm remains a popular approach due to its balance between efficiency and quality of sample distribution.

2.3 Procedural generation of forests

One key motivation for integrating Poisson-Disk Sampling with L-systems is design efficiency, reducing manual workload while maintaining procedural control. In artistic pipelines and game production, manually modeling and placing thousands of trees is labor-intensive and error-prone. As Kenwood et al. [7] demonstrate, leveraging L-system grammars with geometry instancing enabled their system to generate over one million trees in just 4.5 seconds, using roughly 350 MB of memory, a dramatic improvement over naive methods. This illustrates how algorithmic generation can dramatically lower the time and manpower required to populate large scenes, while still offering rich visual variety via cached branching structures.

2.4 Research question

Through this project, we aim to answer the question: *What is the potential of combining L-systems and Poisson-Disk Sampling to generate realistic forests efficiently in terms of design effort and automation?*

3 Implementation

The project's implementation combines Poisson-Disk Sampling based on a tutorial by Sebastian Lague [8] and an L-System tree generator based on a GitHub repository by the user pejoph [9].

The system is designed to generate trees with adjustable parameters such as branching angle, segment length, width, and growth iterations. Inputted parameters can be seen in Figure 2. The program begins with a fixed axiom and applies production rules over multiple iterations to create a string that encodes the tree's structure. Each character in this string corresponds to a drawing command, which is interpreted in 3D space using a turtle graphics metaphor. Custom rule sets define different tree shapes, and users can select among eight predefined variants or generate random ones. Tree segments are rendered using Unity's LineRenderer components, and variance is added to angles for a more organic look.

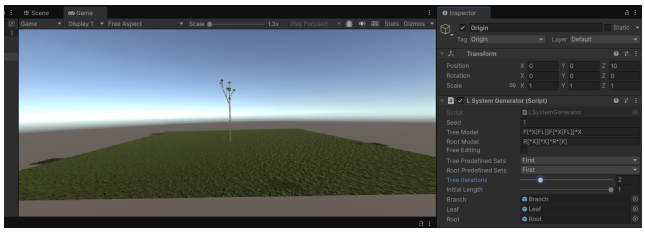

Figure 2: Original L-system tree with parameters

For the Poisson-Disk Sampling, the script generates a collection of points within a defined 2D area such that no two points are closer than a minimum distance, specified by the radius. The algorithm begins by placing a random point in the center of the region and attempts to generate new candidate points around existing ones. Each candidate is generated by picking a random angle and a random distance between 'radius' and '2 * radius' from the current spawn point. Example of generated point can be seen in Figure 3. For each candidate, the 'IsValid' function checks if the point lies within the bounds and is not too close to any other existing point, by referencing a spatial grid that speeds up neighbor searches. If a valid candidate is found, it's added to both the point list and the spawn list, otherwise the spawn point is discarded after a fixed number of failed attempts.

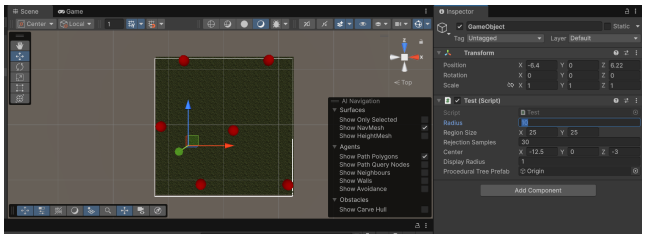

Figure 3: Poisson disc sampling implemented into project

The ‘Test‘ script visualizes the result in the Unity editor using Gizmos, allowing users to see how the distribution changes as parameters like ‘radius‘ or ‘regionSize‘ are adjusted. The generator is reactive to user input through an attached HUD, allowing live updates, regeneration, and resets. This modular design ensures that the system remains flexible, extendable, and interactive.

4 Results

The final implementation successfully generates forest environments composed of L-system trees placed according to Poisson-Disk Sampling. Figure 4 and Figure 5 show two examples of the output produced by the system.

The Poisson-disk algorithm ensures that trees are distributed with a consistent minimum spacing, preventing overlap while maintaining a natural randomness. This spatial distribution results in forest scenes that avoid both clustering and artificial grid-like arrangements.

The L-system component produced visually diverse tree forms depending on the selected rule set and parameters. Even though each tree was generated using deterministic rules, minor variations in rotation, angle noise, and scaling introduced enough diversity to prevent obvious repetition in most scenes.

Overall, the system validates the approach’s efficiency and suitability for real-time applications. The interactive UI allowed live tweaking of parameters, making it useful for iterative design processes or procedural experimentation.

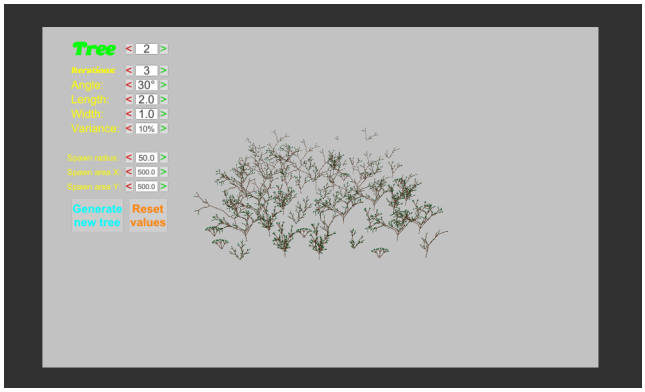

Figure 4: Example of finished output

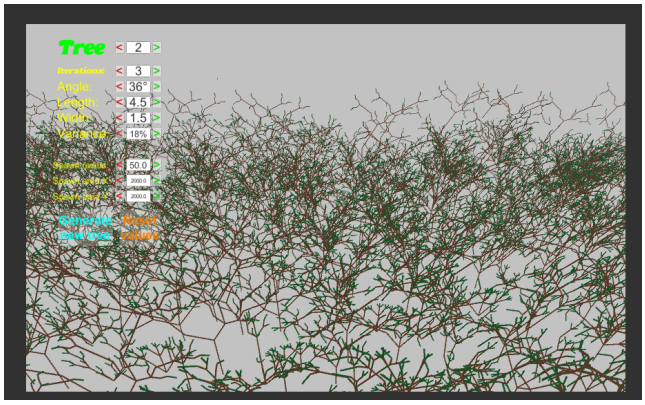

Figure 5: Example 2 of finished output

5 Evaluation

The combined system was evaluated in terms of its ability to efficiently produce visually convincing forest environments without requiring manual placement. From a design efficiency perspective, the tool successfully automates the creation of spatially distributed trees with minimal user input. The Poisson-Disk Sampling algorithm provides even and natural spacing, preventing overlap while avoiding overly regular grid patterns. This ensures believable tree placement that supports the perception of a forest. Meanwhile, the L-System generator introduces organic tree shapes with adjustable parameters, allowing for diverse tree morphologies. The system enables rapid prototyping by letting users tweak distribution and growth parameters in real time through an interactive HUD.

The choice of Fast Poisson-Disk Sampling (Bridson’s algorithm) was motivated by its balance of spatial uniformity and computational efficiency. Despite its slightly non-maximal coverage, it allowed us to scale the system to larger environments without introducing performance bottlenecks.

5.1 Limitations

One key limitation of our approach is the lack of stochasticity in the L-system itself. While deterministic D0L-systems are simple and fast, they can lead to uniformity if not externally randomized. In practice, all trees generated from the same grammar look identical unless post-processing (like rotation or scaling) is applied.

Additionally, the L-system model focuses solely on individual tree structure, not broader ecological interactions. Real forests are shaped by more complex factors such as sunlight competition, terrain features, and species diversity, none of which are captured in our current implementation.

We also noticed some empty spaces in our implementation. It's unclear whether these gaps are due to the Fast Poisson-Disk Sampling method, which, as mentioned earlier, doesn't always fill the space completely, or if there's another issue involved. This would need to be looked into further.

6 Conclusion

This project set out to explore the potential of combining L-systems and Poisson-Disk Sampling for the procedural generation of realistic forests in an efficient and scalable manner. By leveraging Fast Poisson-Disk Sampling, we achieved naturalistic tree placement with consistent spacing, while the L-system component enabled the generation of visually diverse tree structures through rule-based modeling. The modular design, real-time adjustability, and low computational cost of the system highlight its suitability for use in interactive applications such as games or simulations.

While not presenting anything ground-breaking, the combination of these two well-established techniques proved effective in balancing control and automation, significantly reducing manual effort without sacrificing visual quality. The project demonstrates how even a relatively simple system can yield compelling results when designed with flexibility and user interactivity in mind.

Certain limitations remain, particularly the deterministic nature of the L-systems and the lack of ecological complexity, but the results provide a solid foundation for future development. Possible improvements include the integration of stochastic or context-sensitive L-systems, terrain-aware placement, or the simulation of ecological dynamics such as species diversity and growth competition.

Ultimately, the project offers a lightweight and extensible solution for procedural content generation, answering the research question affirmatively and pointing toward interesting directions for future exploration.

References

[1] Benjamin Williams, Panagiotis D. Ritsos, and Christopher Headleand. "Virtual Forestry Generation: Evaluating Models for Tree Placement in Games". In: *Computers* 9.1 (2020). ISSN: 2073-431X. DOI: 10.3390/computers9010020. URL: https://www.mdpi.com/2073-431X/9/1/20.

[2] Gabriela Ochoa and D. Byrnes. *L-Systems: Lecture Notes*. Originally written by Gabriela Ochoa (1998), modified by D. Byrnes. Course notes from the University of Sussex. 2004. URL: http://csweb.wooster.edu/dbyrnes/cs220/lectures/pdffiles/LSystems.pdf (visited on 06/08/2025).

[3] Tong Wang. "Poisson-Disk Sampling: Theory and Applications". In: *Encyclopedia of Computer Graphics and Games*. Cygames, Inc., Tokyo, Japan. Springer, 2024.

[4] Robert L. Cook. "Stochastic Sampling in Computer Graphics". In: *ACM Transactions on Graphics* 5.1 (1986), pp. 51–72. DOI: 10.1145/7529.8927. URL: https://doi.org/10.1145/7529.8927.

[5] Dong Ming Yan et al. "A Survey of Blue-Noise Sampling and Its Applications". In: *Journal of Computer Science and Technology* 30.3 (2015), pp. 439–452. DOI: 10.1007/s11390-015-1535-0.

[6] Robert Bridson. "Fast Poisson disk sampling in arbitrary dimensions". In: *ACM SIGGRAPH 2007 Sketches*. SIGGRAPH '07. San Diego, California: Association for Computing Machinery, 2007, 22–es. ISBN: 9781450347266. DOI: 10.1145/1278780.1278807. URL: https://doi.org/10.1145/1278780.1278807.

[7] Julian Kenwood, James Gain, and Patrick Marais. "Efficient Procedural Generation of Forests". In: *Journal of WSCG* 22.1 (2014), pp. 31–38. URL: http://wscg.zcu.cz/WSCG2014/!!_2014-Journal-No-1.pdf.

[8] Sebastian Lague. *Poisson Disk Sampling*. Accessed: 2025-06-09. YouTube. 2018. URL: https://www.youtube.com/watch?v=7WcmyxyFO7o.

[9] pejoph. *L-Systems*. Accessed: 2025-06-09. GitHub. 2018. URL: https://github.com/pejoph/L-Systems.